CSS do Zero: aprendendo com o Mural de Recados

Um guia prático para quem está começando, com explicações linha por linha e exercícios.

Como usar este material

Este guia foi montado em cima de um projeto real: o **Mural de Recados**, um site com cabeçalho, formulário, lista de recados e rodapé. Em vez de decorar propriedades soltas, você vai entender o *porquê* de cada linha e aplicar no seu próprio código.

Sugestão de estudo: leia um capítulo, digite os exemplos no seu editor (não copie e cole), mude os valores e veja o que acontece no navegador. Errar e testar é a melhor forma de aprender CSS.

Capítulo 1: O que é CSS e como ligar ao HTML

O **HTML** define a *estrutura* da página (títulos, parágrafos, botões). O **CSS** define a *aparência* (cores, tamanhos, espaços, fontes).

Para o navegador usar o seu arquivo CSS, coloque esta linha dentro do `<head>` do HTML:

```
<link rel="stylesheet" href="style.css">
```

- `rel="stylesheet"`: diz que o arquivo ligado é uma folha de estilo.
- `href="style.css"`: o caminho do arquivo. Se ele estiver em outra pasta, o caminho muda (por exemplo, `css/style.css`).

Dica: se o CSS não faz efeito nenhum, quase sempre o problema é o caminho do `href` ou o nome do arquivo escrito diferente.

Capítulo 2: Anatomia de uma regra CSS

Toda regra CSS segue este formato:

```
seletor {
  propriedade: valor;
}
```

- **Seletor**: quem vai receber o estilo.
- **Propriedade**: o que você quer mudar.

- **Valor**: como você quer mudar.
- Cada declaração termina com `;`.

Exemplo:

```
h1 {
  color: darkblue;
  font-size: 2rem;
}
```

Leitura em português: "todo `h1` terá texto azul escuro e tamanho 2rem".

Os três seletores mais usados

Seletor	Exemplo	Seleciona
Tag	`p { }`	todos os parágrafos
Class	`.recado { }`	todo elemento com `class="recado"`
Id	`#envio { }`	o elemento com `id="envio"` (só existe um)

No HTML do Mural:

```
<div class="recado">...</div>
<input type="button" id="envio" value="Envie seu recado">
```

No CSS, o ponto (`.`) indica class e a cerquilha (`#`) indica id.

Regra prática: use **class** para o visual e **id** para o JavaScript. Uma class pode ser repetida em vários elementos; um id deve ser único na página.

Agrupando seletores

A vírgula aplica a mesma regra a vários seletores:

```
h1,
h2,
h3 {
  color: #0a2540;
}
```

Capítulo 3: O modelo de caixa (box model)

Este é o conceito **mais importante** do CSS. Todo elemento é uma caixa, formada por quatro camadas, de dentro para fora:

1. **Content** (conteúdo): o texto ou a imagem.
2. **Padding**: espaço entre o conteúdo e a borda (*dentro*).
3. **Border**: a borda.
4. **Margin**: espaço entre a borda e os outros elementos (*fora*).

```
.caixa {
  width: 300px;
  padding: 20px;
  border: 2px solid #333;
  margin: 30px;
}
```

Padding e margin com vários valores

```
margin: 10px;                /* os 4 lados */
margin: 10px 20px;           /* cima/baixo = 10, esquerda/direita = 20 */
margin: 10px 20px 30px;      /* cima = 10, esq/dir = 20, baixo = 30 */
margin: 10px 20px 30px 40px; /* cima, direita, baixo, esquerda */
```

Para decorar a ordem de 4 valores, pense no relógio: começa em cima e gira no sentido horário.

box-sizing: o salvador

Por padrão, `width: 300px` e `padding: 20px` fazem a caixa ficar com **340px** no total (300 + 20 + 20). Isso bagunça layouts. A solução:

```
* {
  box-sizing: border-box;
}
```

Com `border-box`, o `width` já **inclui** padding e borda. Uma caixa de 300px continua com 300px. Quase todo projeto moderno começa assim.

Capítulo 4: O reset do início do arquivo

Linha por linha do bloco que abre o CSS do Mural:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
```

- `*`: o asterisco seleciona **todos** os elementos da página.
- `margin: 0;`: remove o espaço externo que o navegador coloca por padrão em títulos, parágrafos etc.
- `padding: 0;`: remove o espaço interno padrão.
- `box-sizing: border-box;`: explicado no capítulo anterior.
- `font-family: ...;`: define a fonte. O navegador tenta a primeira (`Segoe UI`); se o computador não tiver, tenta a seguinte, e assim por diante. O `sans-serif` no final é o plano B: qualquer fonte sem serifa.

Por que isso importa? Cada navegador tem seus próprios estilos padrão. O reset zera tudo para você começar do mesmo ponto, em qualquer navegador.

Capítulo 5: Cores e fundos

Existem várias formas de escrever uma cor:

```
color: red;                    /* nome */
color: #0b5ed7;                /* hexadecimal */
color: rgb(11, 94, 215);       /* vermelho, verde, azul (0 a 255) */
color: rgba(0, 0, 0, 0.5);     /* o último valor é a transparência (0 a 1) */
```

Propriedades de cor mais usadas:

- `color`: cor do **texto**.
- `background-color` (ou `background`): cor do **fundo**.
- `border-color`: cor da borda.

O `body` do Mural:

```
body {
  background-color: #f0f2f5;
  color: #333;
  padding: 20px;
}
```

- `background-color: #f0f2f5;`: fundo cinza bem claro.

- `color: #333;` : texto cinza escuro (mais suave que preto puro, o que cansa menos a vista).
- `padding: 20px;` : afasta o conteúdo das bordas da janela.

Variáveis CSS

Se você usa a mesma cor em vários lugares, guarde-a numa variável:

```
:root {
  --azul: #0b5ed7;
  --escuro: #0a2540;
}

.btn {
  background: var(--azul);
}

h1 {
  color: var(--escuro);
}
```

Se um dia você quiser trocar o azul, muda em **um** lugar só.

Capítulo 6: Texto e fontes

```
p {
  color: #555;
  font-size: 1rem;
  font-weight: 600;
  text-align: center;
  line-height: 1.5;
}
```

- `font-size` : tamanho da letra.
- `font-weight` : peso (`normal` , `bold` ou números de 100 a 900).
- `text-align` : alinhamento (`left` , `center` , `right`).
- `line-height` : altura da linha. `1.5` significa 1,5 vez o tamanho da fonte, o que deixa o texto mais respirável.

Unidades: px, rem, % e vh

Unidade	Significado	Quando usar
`px`	pixel, tamanho fixo	bordas, sombras, detalhes
`rem`	múltiplo do tamanho da fonte base (normalmente 16px)	fontes e espaçamentos
`%`	porcentagem do elemento pai	larguras flexíveis
`vh`	1% da altura da tela	seções que ocupam a tela toda

Exemplo: `font-size: 2rem` equivale a 32px na configuração padrão.

Capítulo 7: Bordas arredondadas e sombras

É aqui que nasce o "quadrado meio arredondado".

```
.formulario {
  background: white;
  padding: 20px;
  border-radius: 20px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
```

border-radius

- `border-radius: 10px;` : arredondado discreto.
- `border-radius: 20px;` : bem arredondado.
- `border-radius: 50%;` : em um quadrado, vira um **círculo**.
- `border-radius: 20px 0 20px 0;` : cada canto diferente (topo-esquerda, topo-direita, baixo-direita, baixo-esquerda).

box-shadow

```
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
```

- `0` : deslocamento horizontal.
- `2px` : deslocamento vertical (a sombra desce um pouco).
- `8px` : quanto ela é borrada.
- `rgba(0, 0, 0, 0.1)` : preto com 10% de opacidade.

Experimento: troque `8px` por `30px` e veja a sombra ficar difusa. Troque `0.1` por `0.5` e veja ela ficar forte.

Borda

```
border: 2px solid #ccd3dd;
```

São três informações: espessura, estilo (`solid` , `dashed` , `dotted`) e cor.

Capítulo 8: Centralizando e limitando a largura

O container

```
.container {
  max-width: 450px;
  margin: 0 auto;
}
```

- `.container` : o ponto indica uma class.
- `max-width: 450px;` : a largura **máxima**. Em telas grandes, ele para em 450px; em telas pequenas, encolhe sozinho.
- `margin: 0 auto;` : cima/baixo = 0, esquerda/direita = `auto` . O `auto` divide o espaço sobrando igualmente dos dois lados e **centraliza** a caixa.

Este é o truque que deixa o seu Mural como um "quadrado menor" no meio da tela.

Limitando só o formulário

```
.formulario {
  max-width: 400px;
  margin: 0 auto 25px;
}
```

Com três valores em `margin` , a leitura é: cima = 0, esquerda/direita = auto, baixo = 25px.

Flexbox

O flexbox organiza elementos em linha ou coluna e centraliza com facilidade:

```
.logo {
  display: flex;
  justify-content: center;
}
```

- `display: flex;` : ativa o flexbox no elemento pai.
- `justify-content: center;` : centraliza no eixo principal (horizontal, por padrão).
- `align-items: center;` : centraliza no eixo cruzado (vertical, por padrão).
- `flex-direction: column;` : coloca os itens um embaixo do outro.
- `gap: 15px;` : espaço entre os itens.

Para centralizar algo **na tela inteira**:

```
.tela {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Capítulo 9: Imagens

O logo do Mural estava gigante no começo, porque imagens aparecem no tamanho original. Para controlar:

```
.logo img {
  width: 90px;
  height: auto;
}
```

- `.logo img` : seleciona apenas as `<img>` que estão **dentro** de um elemento com class `logo` .
- `width: 90px;` : define a largura.
- `height: auto;` : a altura se ajusta sozinha, mantendo a proporção (a imagem não fica esticada).

Outras propriedades úteis:

```
img {
  max-width: 100%;      /* nunca passa do tamanho do pai */
  object-fit: cover;    /* preenche a área cortando o excesso, sem distorcer
*/
  border-radius: 50%;   /* imagem redonda */
}
```

Capítulo 10: Formulários

Formulários têm algumas manhas que valem um capítulo.

Labels

```
label {
  display: block;
  font-weight: 600;
  margin: 12px 0 6px;
}
```

- `display: block;`: faz o label ocupar a linha toda, então o campo vai para a linha de baixo (você não precisa de `<br>`).
- `margin: 12px 0 6px;`: 12px em cima, 0 nos lados, 6px embaixo.

Importante no HTML: o `for` do label deve ser igual ao `id` do campo.

```
<label for="nome">Seu nome:</label>
<input type="text" id="nome" name="nome">
```

Assim, clicar no texto do label coloca o cursor no campo.

Campos de texto

```
input[type="text"],
textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccd3dd;
  border-radius: 8px;
  font-size: 1rem;
}
```

- `input[type="text"]`: seleciona apenas inputs do tipo texto. Esse formato se chama **seletor de atributo**.
- `width: 100%;`: o campo ocupa toda a largura disponível.
- `padding: 10px;`: o texto não fica colado na borda.

Textarea

```
textarea {
  height: 100px;
  resize: vertical;
}
```

- `resize: vertical;`: permite o usuário esticar só para cima e para baixo (evita quebrar o layout esticando para os lados).

Estado de foco (:focus)

```
input[type="text"]:focus,
textarea:focus {
  outline: none;
  border-color: #0b5ed7;
  box-shadow: 0 0 0 3px rgba(11, 94, 215, 0.2);
}
```

- `:focus` é uma **pseudo-classe**: aplica o estilo quando o campo está selecionado.
- `outline: none;`: remove o contorno padrão do navegador.
- A `box-shadow` com `0 0 0 3px` cria um "halo" azul ao redor, e como substituímos o contorno padrão, é importante manter esse destaque para quem navega pelo teclado.

Herdando a fonte

```
input,
textarea,
button {
  font-family: inherit;
}
```

Campos de formulário **não herdam** a fonte do site por padrão: o navegador usa a dele (por isso o seu textarea aparecia em fonte monoespaçada). O `inherit` manda copiar a fonte do elemento pai.

O botão

```
#envio {
  margin-top: 15px;
  width: 100%;
  background: #0b5ed7;
  color: white;
  border: none;
  padding: 12px;
  border-radius: 8px;
  font-size: 1rem;
  cursor: pointer;
}

#envio:hover {
  background: #094db0;
}
```

- `border: none;`: tira a borda padrão do botão.
- `cursor: pointer;`: o mouse vira uma mãozinha.
- `:hover`: pseudo-classe que aplica o estilo quando o mouse está em cima.

Dica: para o texto do botão `<input type="button">`, use o atributo `value`. Se quiser colocar negrito, ícones ou outras tags dentro, use `<button>` no lugar.

Capítulo 11: Montando o cartão de recados

O bloco do formulário e cada recado compartilham o mesmo visual de "cartão branco":

```
.formulario,
.recado {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
  margin-bottom: 25px;
}
```

Repare na vírgula: um bloco só estiliza dois seletores. Isso evita repetir código.

Detalhes do recado:

```
.data {
  font-size: 0.85rem;
  color: #888;
}

.recado-texto {
  margin-top: 8px;
}
```

Capítulo 12: O CSS completo do Mural

Juntando tudo o que vimos:

```
/* Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

input,
textarea,
button {
  font-family: inherit;
}

body {
  background-color: #f0f2f5;
  color: #333;
  padding: 20px;
}

.container {
  max-width: 450px;
  margin: 0 auto;
}

/* Logo */
.logo {
  display: flex;
  justify-content: center;
  margin-bottom: 15px;
}
```

```
.logo img {
  width: 90px;
  height: auto;
}

/* Cabeçalho */
.cabecalho {
  text-align: center;
  margin-bottom: 25px;
}

.cabecalho h1 {
  font-size: 2rem;
  color: #0a2540;
  margin-bottom: 8px;
}

.cabecalho p {
  color: #555;
  line-height: 1.5;
}

.linha {
  border: none;
  height: 2px;
  background: #d0d7e2;
  margin-top: 20px;
}

/* Subtítulos */
.subtitulo h2 {
  font-size: 1.3rem;
  color: #0a2540;
  margin-bottom: 12px;
}

/* Cartões */
.formulario,
.recado {
  background: white;
  padding: 20px;
  border-radius: 20px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
  margin-bottom: 25px;
}
```

```
/* Formulário */
label {
  display: block;
  font-weight: 600;
  margin: 12px 0 6px;
}

input[type="text"],
textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccd3dd;
  border-radius: 8px;
  font-size: 1rem;
}

textarea {
  height: 100px;
  resize: vertical;
}

input[type="text"]:focus,
textarea:focus {
  outline: none;
  border-color: #0b5ed7;
  box-shadow: 0 0 0 3px rgba(11, 94, 215, 0.2);
}

#envio {
  margin-top: 15px;
  width: 100%;
  background: #0b5ed7;
  color: white;
  border: none;
  padding: 12px;
  border-radius: 8px;
  font-size: 1rem;
  cursor: pointer;
}

#envio:hover {
  background: #094db0;
}

/* Recados */
.data {
```

```
  font-size: 0.85rem;
  color: #888;
}

.recado-texto {
  margin-top: 8px;
}

/* Rodapé */
.rodape {
  text-align: center;
  color: #555;
  margin-top: 30px;
}
```

Capítulo 13: Erros comuns no HTML (e como corrigir)

No código do Mural apareceram alguns detalhes que valem como lição:

1. **`for` diferente do `id`.** Se o label diz `for="name"` e o input tem `id="nome"`, a ligação não funciona. Os dois precisam ser iguais.
2. **Atributo no lugar errado.** `rows="4"` pertence ao `<textarea>`, não ao botão.
3. **Nome com espaços.** `name="Escreva seus avisos"` pode dar problema. Prefira `name="aviso"`.
4. **`<br>` demais.** Com `label { display: block }`, você não precisa deles dentro do formulário.
5. **Arquivos com acento e espaço.** `Ícone_de_Nota_Fixada...png` pode falhar em alguns servidores. Renomeie para algo simples, como `logo.png`.
6. **Ordem das regras.** No CSS, se duas regras competem, a que vem **depois** vence (quando têm a mesma força). Se um estilo não aparece, procure se não há outra regra sobrescrevendo mais abaixo.

Capítulo 14: Como depurar quando algo não funciona

1. Aperte **F12** no navegador para abrir as ferramentas do desenvolvedor.
2. Clique no ícone de seta (selecionar elemento) e clique na parte da página que está errada.
3. No painel **Styles**, veja quais regras estão sendo aplicadas. Regras riscadas foram sobrescritas por outras.
4. Você pode editar valores ali mesmo para testar, sem mexer no arquivo.

5. Clique no ícone de celular para testar em tela pequena.

Esse hábito sozinho economiza horas de tentativa e erro.

Capítulo 15: Exercícios

Tente fazer sem olhar as respostas acima.

1. Crie uma `div` com class `cartao`, 250px de largura, fundo amarelo claro, borda de 2px tracejada e cantos arredondados de 15px.
2. Faça uma imagem circular de 120px por 120px, sem distorcer.
3. Centralize um título no meio da tela, na horizontal e na vertical, usando flexbox.
4. Faça um botão que fique verde normalmente e verde-escuro quando o mouse passar por cima.
5. Troque todas as cores azuis do Mural por variáveis CSS e depois mude o tema para verde alterando uma linha só.
6. Desafio: faça cada recado ter uma borda esquerda colorida de 5px (`border-left`).

Capítulo 16: Folha de consulta rápida

Quero...	Use
mudar a cor do texto	`color`
mudar a cor de fundo	`background`
espaço dentro da caixa	`padding`
espaço fora da caixa	`margin`
cantos arredondados	`border-radius`
círculo	`border-radius: 50%` em um quadrado
sombra	`box-shadow`
centralizar uma caixa	`margin: 0 auto` + `max-width`
centralizar o conteúdo	`display: flex` + `justify-content: center`
centralizar texto	`text-align: center`
colocar itens em coluna	`flex-direction: column`
espaço entre itens	`gap`
mouse em cima	`:hover`

Quero...	Use
campo selecionado	`:focus`
mãozinha no botão	`cursor: pointer`
largura que cabe no padding	`box-sizing: border-box`

Próximos passos

Depois de dominar este material, siga esta ordem:

1. **Responsividade**: `@media (max-width: 600px) { ... }` para adaptar o layout a celulares.
2. **CSS Grid**: para layouts em duas dimensões.
3. **JavaScript**: fazer o botão "Envie seu recado" criar um novo recado na lista, lendo o nome e o texto dos campos.
4. **Armazenamento**: guardar os recados no navegador (`localStorage`) para não sumirem ao recarregar a página.

Bons estudos!